

情報科学演習 --- 発表・質疑応答対策

ABC Music Suite システム理解ガイド

～ モジュール構成・役割解説・想定問答集 ～

本書は、開発した「ABC Music Suite」のモジュール構成、各ファイルが担当する役割、および発表会で想定される質疑応答への対策をまとめたガイドブックです。公式の技術仕様書 ([main.pdf](#)) を補完し、自分たちの言葉でシステムを説明できるようにするための理解支援資料として活用してください。

目次

1	発表会・質疑応答での想定問答集	2
1.1	Q1: サーバーを FastAPI と Node.js の 2 つに分けている理由は？	2
1.2	Q2: ChatGPT 等の外部 API ではなく、ローカル LLM (Ollama) を選んだ理由は？	2
1.3	Q3: なぜ MIDI データを直接生成せず「ABC 記法」を経由させるのか？	3
1.4	Q4: Raspberry Pi とホスト PC はどのように連携しているのか？	3
2	モジュール構成とディレクトリ構造	5
2.1	ディレクトリ構造マップ	5
2.2	主要モジュールの「働き」と関係性	6
3	シンプルなシステム相関・データ連携図	7
3.1	データの流れ（自動作曲の例）	7
4	発表でアピールすべき設計の工夫点	8
4.1	疎結合設計（ルーズカップリング）の適用	8
4.2	リソースの適材適所配置（エッジ・ホスト PC 構成）	8
4.3	バリデーションフィルターと AI 自己修復リトライ	8
4.4	ライセンスと保証免責（メーカーとしての責任）	9

1 発表会・質疑応答での想定問答集

審査員や他の学生から質問されやすい「技術選定の意図」や「システムの境界」について、情報科学的に説得力のある切り返しを整理しました。

1.1 Q1: サーバーを FastAPI と Node.js の 2 つに分けている理由は？

質問の要旨

「なぜ Python (FastAPI) と Node.js (Express) の 2 つのサーバーを起動しているのですか？ Python のライブラリだけで完結できなかったのですか？」

【回答案】

「MIDI と楽譜テキスト (ABC 記法) の相互変換には、実績のある C 言語製のコマンドラインツール `abc2midi` および `midi2abc` を使用しています。これらを Web API として安全にラッピングするにあたり、先行の安定したオープンソースプロジェクトである `abc2midiGUI` (Node.js/Express 製) の資産をそのまま再利用 (プロキシ通信) する方針をとりました。

これを Python に無理やり移植すると、プロセスの並行実行処理や環境依存のバグが発生するリスクがありました。そのため、本システムでは『**マイクロサービスの設計思想**』に基づき、MIDI 変換処理を Node.js サーバーにカプセル化 (専門特化) させ、メインの FastAPI バックエンドから HTTP 経由で呼び出す形にしました。これにより、各サーバーコードが非常にシンプルかつ頑健に保たれています。」

1.2 Q2: ChatGPT 等の外部 API ではなく、ローカル LLM (Ollama) を選んだ理由は？

質問の要旨

「API キーを取得して ChatGPT や Claude を使った方が賢い音楽が作れるのでは？ なぜわざわざ重いローカル LLM を動かすのですか？」

【回答案】

「理由は大きく 2 つあります。1 つ目は**完全なローカル（オフライン）環境での動作保証とセキュリティ**です。本システムは LAN 内で完結するように設計されており、外部のインターネット回線の品質や切断に影響されずに動作します。

2 つ目は**ランニングコストがゼロ**である点です。外部 API はリクエストごとに課金され、大量の検証やループ再生テストでコストが膨らみます。ローカル LLM を利用することで、開発中のテスト回数に制限がなくなり、かつ Raspberry Pi とローカル PC 間のクローズドな無線 LAN 環境だけで安定したエッジ AI 連携を実現することができました。」

1.3 Q3: なぜ MIDI データを直接生成せず「ABC 記法」を経由させるのか？

質問の要旨

「AI に直接 MIDI や WAV のバイナリデータを作らせないのはなぜですか？ なぜテキストの ABC 記法 を挟む必要があるのですか？」

【回答案】

「大規模言語モデル（LLM）は、本質的に『テキスト（文字配列）の次に来る確率の高い単語を予測する』仕組みです。そのため、0 と 1 で構成されるバイナリデータ（MIDI ファイルや音声 WAV データ）を直接出力することは非常に困難です。

一方、ABC 記法は**楽譜をテキスト（例：ドレミは c d e）で表現する規格**です。LLM にとって ABC 記法は『楽譜言語という外国語のテキスト』として処理できるため、文脈を捉えた破綻のないメロディを容易に生成できます。AI が生成したテキスト楽譜を、バックエンド側の Python と Node.js サーバーで MIDI / WAV に安全に波形合成するプロセスをとることで、AI の強みとプログラムの確実性を組み合わせています。」

1.4 Q4: Raspberry Pi とホスト PC はどのように連携しているのか？

質問の要旨

「Raspberry Pi と PC の間では、具体的に何が送受信され、どこで音が鳴っているのですか？」

【回答案】

「同じ Wi-Fi ルーター（LAN）に接続された環境下で、ラズパイからホスト PC の接続状況をテストするために `test_connection.py` から HTTP リクエストを送信することができます。

ユーザーは、ラズパイにコピーした `play_abc.py` を使用して、PC 等で作成された ABC 楽譜ファイルを直接読み込んだり、楽譜テキストを引数に指定して自動演奏させることができます。重い AI 推論や WAV 生成は PC 側で行い、ラズパイはスピーカーを鳴らす音声出力（エッジデバイス）として協調する構成です。」

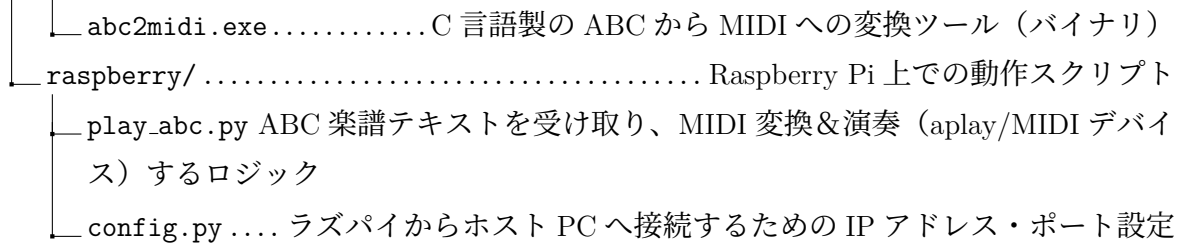
2 モジュール構成とディレクトリ構造

本システムのコードベースは、機能ごとの役割が明確に分離されています。以下にディレクトリマップと、各ファイルが担当する「働き」を示します。

2.1 ディレクトリ構造マップ

```

abc2midiGUI.....プロジェクトルート (c:/Users/yaito/Downloads/abc2midiGUI)
├─ start.ps1.....システム全体を一括起動するコントロールスクリプト (PowerShell)
├─ start.bat.....一括起動用の Windows バッチファイル
├─ host.ps1.....ホスト PC の IP アドレス等を設定するアシストスクリプト
├─ copy-to-pi.ps1.....Raspberry Pi へファイルを同期するためのスクリプト
├─ abc_synthesizer.py ABC テキストを解析し、プログラマ的に WAV 音声合成するモジュール
├─ server/.....Python FastAPI バックエンドサーバー
│   ├── main.py.....FastAPI サーバーの起動、CORS 設定、静的ファイル配信登録
│   ├── requirements.txt.....Python バックエンドの依存ライブラリ指定
│   └── modules/.....サーバー内共通処理
│       ├── config.py Ollama や MIDI サーバーの接続先 URL、保存ディレクトリ等の設定管理
│       └── llm_client.py Ollama API と通信し、プロンプトの送信やストリーム受信を行う
│   └── routes/.....各 API エンドポイントの実装
│       ├── compose.py.....自動作曲リクエストの受付・楽譜抽出・WAV 合成処理を担当
│       ├── harmonize.py.....既存メロディへの自動伴奏リクエストを担当
│       ├── chat.py.....AI との汎用対話（チャット）エンドポイント
│       └── midi_convert.py.....MIDI と ABC 記法の相互変換リクエストを担当
├─ web-ui/.....React フロントエンド（Vite 構成）
│   └── src/.....フロントエンドソースコード
│       ├── App.jsx.....メイン UI 画面、グローバル状態、タブ切り替え（4 機能）を管理
│       ├── index.css 全体デザインシステム（ダークモード、グラデーション、フォント）
│       └── components/.....各タブの画面コンポーネント
│           ├── ComposeTab.jsx...自動作曲タブ画面（テーマ入力、WAV 再生、楽譜表示）
│           ├── HarmonizeTab.jsx...自動伴奏タブ画面（メロディ入力、コード付与指示）
│           ├── MidiConvertTab.jsx..MIDI/ABC 相互変換タブ画面（ファイルドラッグ&ドロップ, 変換）
│           └── ChatTab.jsx.....AI 音楽チャットタブ画面（対話、楽譜自動演奏）
└─ midi-server/.....Node.js MIDI 変換中継プロキシ
    └─ server.js.....Express で構築された MIDI・ABC 相互変換 API
  
```



2.2 主要モジュールの「働き」と関係性

1. **UI 層 (web-ui)**: ブラウザ上で動作し、ユーザーの入力を受け付けます。FastAPI サーバーの API に非同期（Fetch API / EventSource）で通信し、結果（楽譜や WAV の URL）を可視化します。
2. **API・ロジック層 (server)**: FastAPI が全リクエストの交通整理を行います。自動作曲時は `modules/llm_client.py` を通じて Ollama から楽譜を取得し、`abc_synthesizer.py` でそれを音声（WAV）化してブラウザに URL を返します。
3. **変換仲介層 (midi-server)**: バイナリ（`abc2midi.exe` 等）を直接 Python から叩くのではなく、Node.js Express を経由した「Web API」として外出ししています。これにより、変換サーバーを別 PC に配置するなどのスケーラビリティが確保されています。
4. **エッジ出力層 (raspberry)**: Ollama や FastAPI から楽譜テキストを受け取り、ラズパイのサウンドカードを介して物理的なスピーカーから演奏します。

3 シンプルなシステム相関・データ連携図

システムの全体像と、データがどのように流れるか（どのポートを使ってどのファイル同士が連携しているか）を直感的にまとめた相関図を図 1 に示します。

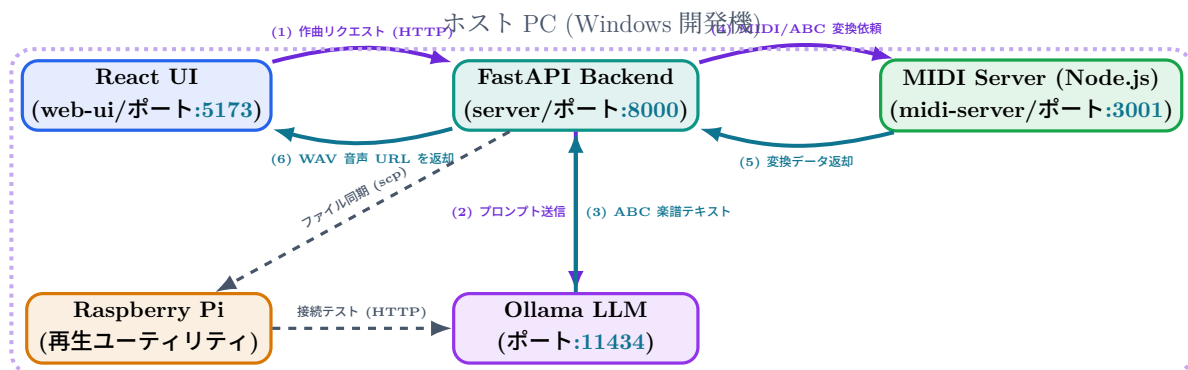


図 1 ABC Music Suite 各モジュールの連携関係とデータの流れ

3.1 データの流れ（自動作曲の例）

図 1 における自動作曲時のデータ推移は以下の通りです：

- (1) リクエスト**：ブラウザ（React UI）から作曲テーマ「明るい朝の曲」とパラメータが FastAPI に届きます。
- (2) AI 推論要求**：FastAPI は適切なプロンプト（指示書）を組み立てて Ollama に送ります。
- (3) 楽譜生成**：Ollama は ABC 記法（楽譜テキスト）を生成し、FastAPI に返します。
- (4)・(5) MIDI 変換**：FastAPI は生成された ABC 記法を Node.js (MIDI Server) に渡し、MIDI ファイルへと構造化します。
- WAV 波形合成**：FastAPI は `abc_synthesizer.py` を動かし、MIDI/ABC データをもとに WAV 音声データを生成・保存します。
- (6) レスポンス**：合成された WAV の URL と、ABC 楽譜テキストがブラウザに返され、ブラウザ側で即座に演奏・可視化されます。

4 発表でアピールすべき設計の工夫点

このシステムが「単に組み合わせただけ」ではなく、情報科学的な考慮に基づいて設計されていることを示すアピールポイントです。

4.1 疎結合設計（ルーズカップリング）の適用

モジュール同士の依存度を極限まで低くすることで、将来の改修に非常に強いシステムとなっています。

アピール例: AI エンジンの差し替え

「現在、AI 推論には Ollama を使用していますが、将来的に ChatGPT (OpenAI API) や Claude に移行する場合でも、変更が必要なのは `server/modules/llm_client.py` のクラス内部だけです。フロントエンド (UI) や、FastAPI の各ルート (エンドポイント) コードは 1 行も書き換える必要がありません。」

4.2 リソースの適材適所配置（エッジ・ホスト PC 構成）

スペックの異なるデバイスをネットワーク越しに協調させる「適材適所」の分散システム設計を実践しています。

アピール例: ラズパイの役割分担

「Raspberry Pi のような非力なシングルボードコンピュータで直接大規模 LLM（数 GB のモデル）を動かすことは、メモリや処理速度の面で現実的ではありません。そこで、推論や音声合成などの『重い計算』はすべてホスト PC の GPU リソースに任せ、ラズパイはユーザー入力をホスト PC に届けてスピーカーで再生するだけの『エッジデバイス』に徹するように設計しました。これにより、デバイスの限界を超えた高度な AI 連携が低消費電力で実現できています。」

※また、ラズパイ側の `raspberrypi/play_abc.py` が直接ホスト PC の音声ファイルをダウンロードしてローカルで MIDI 再生する仕組みにより、物理的なスピーカー構成への切り替えも容易です。

4.3 バリデーションフィルターと AI 自己修復リトライ

AI が出力した楽譜テキストが正しい文法や拍数に従っているかをシステム側で検証し、エラーがあれば自動で修正指示を投げる「自己修復ループ」を組み込んでいます。

アピール例: AI への自動差し戻し

「大規模言語モデル（LLM）は稀に小節の拍数を間違えたり、ABC 記法で使えない無効な文字を出力することがあります。本システムでは、バックエンド側に『バリデーター（`abc_validator.py`）』を配置し、楽譜ヘッダー・使用文字・拍数（4 拍子）を自動で数学的に検証しています。

もしエラーを検知した場合、単に失敗を返すのではなく、エラー内容（例: 『1 小節目が 4.5 拍です』）を AI にフィードバックし、最大 5 回まで自動で差し戻して再生成（リトライ）させます。この検証&フィードバックループにより、AI の文法ミスによる音声合成のクラッシュを防ぎ、極めて高い生成成功率を実現しています。」

4.4 ライセンスと保証免責（メーカーとしての責任）

ソフトウェアの配布や利用における権利関係・セキュリティ上の責任範囲を明確に定義しています。

アピール例: 免責事項の明記

「本システムは、LAN 内でポートを解放してラズパイからアクセスを受ける構造になっています。開発者（メーカー）としての責任範囲を切り分け、意図しないポート開放による不正アクセスやハードウェアの故障に対する免責事項を MIT ライセンスと合わせて明記しました。これにより、ソフトウェアの公開における法的な管理手順を実践的に学んでいます。」